Multiuser Environment

CPE 241 RDBMS
Week 9

Agenda

Transaction

Assertion & Trigger

Permission

View

Transaction

a sequence of operations performed as a single logical unit of work

Includes many operations (insert, update, delete)

Key Properties of Transactions: ACID

1. **Atomicity** – All operations in a transaction are completed successfully, or none are.
2. **Consistency** – A transaction takes the database from one valid state to another.
3. **Isolation** – Transactions are isolated from one another until they are finished.
4. **Durability** – Once a transaction is committed, the changes are permanent, even in case of a system failure.

Common Read Anomalies

Dirty Read

You read data that another transaction hasn't committed yet (and might roll back).

➤ Only allowed in **Read Uncommitted**.

Non-Repeatable Read

You read the same row twice, and the value is different because another transaction updated it in between.

➤ Prevented from **Repeatable Read** upwards.

Phantom Read

You run a query twice (e.g., `SELECT * FROM orders WHERE price > 1000`), and the second time, **new rows appear** that match the criteria.

➤ Only prevented by **Serializable**.

Four Isolation Levels (ANSI SQL Standard)

Isolation Level	Allows Dirty Read	Allows Non-Repeatable Read	Allows Phantom Read
Read Uncommitted	✅ Yes	✅ Yes	✅ Yes
Read Committed	❌ No	✅ Yes	✅ Yes
Repeatable Read	❌ No	❌ No	✅ Yes
Serializable	❌ No	❌ No	❌ No

Read Uncommitted: You're peeking into inventory even while someone else is changing it — risky but fast.

Read Committed: You only see finalized inventory updates.

Repeatable Read: You see the same stock levels no matter how many times you look during your task.

Serializable: You lock the whole warehouse — nobody else can update or even insert until you finish.

Autocommit

In many RDBMS (like MySQL), **autocommit is enabled by default**. This means:

- Every individual SQL statement (like `INSERT`, `UPDATE`, `DELETE`) is treated as a separate transaction.
- It is **automatically committed** immediately after it runs.
- You cannot `ROLLBACK` because the change is already saved.

To use **`START TRANSACTION`**, `SAVEPOINT`, `ROLLBACK`, and `COMMIT` effectively, you need to **turn off autocommit**, otherwise your statements are committed one by one.

MySQL

```
SET autocommit = 0;

START TRANSACTION;

-- do something

COMMIT; -- or ROLLBACK;

SET autocommit = 1;
```

Basic Transaction Operations

Operation	Description
`BEGIN` / `START TRANSACTION`	Starts a new transaction.
`COMMIT`	Saves all the changes made in the transaction permanently.
`ROLLBACK`	Undoes all the changes made in the transaction (back to the last COMMIT or beginning).
`SAVEPOINT`	Sets a point within a transaction to which you can later roll back.
`ROLLBACK TO SAVEPOINT`	Rolls back to a specific savepoint.
`SET TRANSACTION ISOLATION LEVEL`	Sets the isolation level for the current transaction.

Examples

```
-- Start the transaction
START TRANSACTION;

-- Step 1: Deduct from Account A
UPDATE accounts SET balance = balance - 500 WHERE account_id = 1;

-- Step 2: Add to Account B
UPDATE accounts SET balance = balance + 500 WHERE account_id = 2;

-- If all good, commit the transaction
COMMIT;

ROLLBACK;  -- Undoes both UPDATEs if error occurs
```

Savepoint

```
START TRANSACTION;

UPDATE products SET stock = stock - 10 WHERE id = 101;

SAVEPOINT sp1;

UPDATE products SET stock = stock - 5 WHERE id = 102;

-- Suppose we want to undo only the second update

ROLLBACK TO sp1;

-- Commit the rest

COMMIT;
```

ISOLATION LEVEL

```
SET TRANSACTION ISOLATION LEVEL REPEATABLE READ;

START TRANSACTION;

-- Your queries here

COMMIT;
```

How Locks Work with Transactions

Isolation Level	Locking Behavior
Read Uncommitted	No locks → allows dirty reads
Read Committed	Shared locks for read, exclusive for write, released after each query
Repeatable Read	Shared locks remain for the entire transaction
Serializable	Strictest → full range locks to simulate serial execution

Lock Type	Description
Shared Lock (S Lock)	Allows **reading** data. Other transactions can also read but not write.
Exclusive Lock (X Lock)	Allows **writing** data. No other transaction can read or write the locked data.
Row-level Lock	Locks a single row (more concurrent-friendly).
Table-level Lock	Locks the entire table (simpler but less concurrency).

Locking Behavior in Transactions

Operation	Lock Type Applied
`SELECT`	No lock or **Shared Lock**, depending on isolation level
`SELECT ... FOR UPDATE`	**Exclusive Lock** (row-level)
`INSERT`, `UPDATE`, `DELETE`	**Exclusive Lock** on affected rows
`SELECT` at `SERIALIZABLE` level	**Range locks** or **table locks** to prevent phantom reads

Assertion and Trigger

Assertion is a **database constraint** that ensures a **condition is always true**

Do we know what a constraint is? Constraint is on its own table, Assertion can be on other tables

```
CREATE ASSERTION check_salary_limit
CHECK (
  NOT EXISTS (
    SELECT * FROM employees
    WHERE salary > 100000
  )
);
```

****But many dbms does not support assertion!**

Check Constraint

```
CREATE TABLE employee (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    position VARCHAR(50),
    salary DECIMAL(10,2),
    CHECK (salary <= 100000)  -- Works in MySQL 8.0+ only
);
```

Trigger in MySQL

A **Trigger** is a piece of code that is **automatically executed in response to a specific event** on a table (such as `INSERT`, `UPDATE`, `DELETE`).
Triggers help **enforce rules** and **automate behavior** at the database level.

```
CREATE TRIGGER salary_check_before_insert
BEFORE INSERT ON employees
FOR EACH ROW
BEGIN
  IF NEW.salary > 100000 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Salary exceeds the limit!';
  END IF;
END;
//
```

Trigger Timing & Event

Type	When it Executes
BEFORE	Before the actual operation is executed. Useful for validation or modification.
AFTER	After the operation is completed. Useful for logging or auditing.

Type	Description
INSERT	Trigger fires when a new row is inserted.
UPDATE	Trigger fires when a row is updated.
DELETE	Trigger fires when a row is deleted.

Example of Each Trigger in MySQL

```
CREATE TRIGGER trg_before_insert_employee

BEFORE INSERT ON employee

FOR EACH ROW

BEGIN

  -- Modify or validate data before insert

  IF NEW.salary > 100000 THEN

    SIGNAL SQLSTATE '45000'

    SET MESSAGE_TEXT = 'Salary too high!';

  END IF;

END;
```

AFTER INSERT

```
CREATE TRIGGER trg_after_insert_employee

AFTER INSERT ON employee

FOR EACH ROW

BEGIN

  INSERT INTO employee_log (emp_id, action, log_time)

  VALUES (NEW.id, 'INSERTED', NOW());

END;
```

MySQL Limitations

MySQL supports **only row-level triggers** (no statement-level triggers).

You **cannot create multiple triggers of the same type on the same table**. (e.g., only one `BEFORE INSERT` per table)

Triggers **can't call `COMMIT` or `ROLLBACK`**.

Privileges

Permission

Privilege	Description
`SELECT`	Read data
`INSERT`	Add new data
`UPDATE`	Modify existing data
`DELETE`	Remove data
`CREATE`	Create tables or databases
`DROP`	Delete tables or databases
`ALTER`	Modify structure of tables
`ALL PRIVILEGES`	All of the above

Example

```
-- Create a new user
CREATE USER 'student'@'localhost' IDENTIFIED BY 'studentpass';

-- Grant SELECT and INSERT on a specific table
GRANT SELECT, INSERT ON mydb.employee TO 'student'@'localhost';

-- To grant all privileges on a database
GRANT ALL PRIVILEGES ON mydb.* TO 'adminuser'@'localhost';

-- Revoke privilege
REVOKE INSERT ON mydb.employee FROM 'student'@'localhost';

FLUSH PRIVILEGES;
```

Views

A **View** is a **virtual table** based on the result of an SQL query. It **does not store data itself**, but provides a way to:

- Simplify complex queries
- Restrict access to certain columns or rows (for security)
- Present data in a specific format
- Not storing data
- Access data from original table
- Can be used like a table

```
CREATE VIEW view_name AS
SELECT columns
FROM tables
WHERE conditions;

UPDATE employee_public SET position = 'Team Lead' WHERE id = 1; -- can be updated if only one table
GRANT SELECT ON employee_public TO 'student'@'localhost'; -- can grant privilege like table
```

Example: Hide Salary in public view

```
CREATE TABLE employee (
  id INT PRIMARY KEY AUTO_INCREMENT,
  name VARCHAR(100),
  position VARCHAR(50),
  salary DECIMAL(10,2)
);
```

```
CREATE VIEW employee_public
AS
SELECT id, name, position
FROM employee;
```

Role-based View – Sales Manager

```
CREATE TABLE employee (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    position VARCHAR(50),
    department VARCHAR(50),
    salary DECIMAL(10,2),
    hire_date DATE
);
```

```
CREATE VIEW sales_team_view AS
SELECT id, name, position, department,
hire_date
FROM employee
WHERE department = 'Sales';
```

Dynamic Role-Based View Access

```
CREATE TABLE employee (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    position VARCHAR(50),
    department VARCHAR(50),
    salary DECIMAL(10,2)
);
```

```
CREATE TABLE manager (
    username VARCHAR(50) PRIMARY KEY,
    full_name VARCHAR(100),
    department VARCHAR(50)
);
```

```
INSERT INTO manager (username, full_name, department)
VALUES ('sales_manager', 'Jane Smith', 'Sales');
```

```
CREATE VIEW department_employee_view AS
SELECT e.id, e.name, e.position, e.department
FROM employee e
JOIN manager m ON e.department = m.department
WHERE CURRENT_USER() = CONCAT(m.username,
'@localhost');
```

```
CREATE USER 'sales_manager'@'localhost' IDENTIFIED BY 'pass123';
GRANT SELECT ON department_employee_view TO
'sales_manager'@'localhost';
```

- No need to create a new view per department
- No need to hardcode department names into views
- No access to base `employee` or `manager` tables

Other options

```
CREATE TABLE employee (
    id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    position VARCHAR(50), -- e.g., 'Manager', 'Engineer'
    department VARCHAR(50),
    salary DECIMAL(10,2),
    is_manager BOOLEAN DEFAULT FALSE
);
```

Pros:

Simpler schema (only one table)
Easier to update shared fields (e.g., salary, name, department)
You can easily filter managers with `WHERE is_manager = TRUE`

Cons:

May not scale well if managers need more metadata (e.g., usernames, roles, permissions)
Harder to decouple business logic for manager-specific access

```
CREATE TABLE manager (
    emp_id INT PRIMARY KEY,
    username VARCHAR(50) UNIQUE,
    FOREIGN KEY (emp_id) REFERENCES employee(id)
);
```

Pros:

Clean separation of manager identity and login
Easier to store **login credentials**, **usernames**, and role-based metadata
Supports more flexible role-based security setup (especially for views)

🔍 **Cons:**

Slightly more complex queries (may need joins)
You must ensure `emp_id` exists in `employee`

Generalized Role-based View

```
CREATE VIEW department_employee_view AS
SELECT e.id, e.name, e.position, e.department
FROM employee e
JOIN manager m ON e.department = (SELECT department FROM employee WHERE id = m.emp_id)
WHERE CURRENT_USER() = CONCAT(m.username, '@localhost');
```

`SELECT ... FOR UPDATE` in Views

What it does:

`SELECT ... FOR UPDATE` is used inside a transaction to **lock the selected rows**, so other transactions can't update them until the current one is committed or rolled back.

❗ But in MySQL:

You **cannot use `FOR UPDATE` on views directly** unless:

- The view is **updatable**
- It is based on **one base table**
- It doesn't include `DISTINCT`, `GROUP BY`, joins, or aggregate functions

```
START TRANSACTION;

-- This works if the view is updatable
SELECT * FROM sales_team_view WHERE id = 4 FOR UPDATE;

-- Modify something
UPDATE employee SET position = 'Senior Sales' WHERE id = 4;

COMMIT;
```